

Security Basics

Video 4 of "REST API Mastery" series — Easy English study notes to help you understand and explain REST APIs confidently in interviews.

Backend

MERN

Interview Prep

HTTP Basic Authentication

Basic Authentication is the simplest way to protect an API. Every single time the client sends a request, it also sends a username and password with it.

The username and password are joined together, then changed into a coded text (this is called Base64). This coded text is placed inside the request.

```
Authorization: Basic dXNlcjpwYXNz
```

Important: This coding is NOT locking the data safely. Anyone can turn it back into normal text in 2 seconds. So this method is only safe when used together with HTTPS (the safe, locked version of a website connection).

Easy Analogy: It's like showing your ID card at every single door of a building, every time you want to go in. It works, but it is an old method. And if someone secretly watches you show the card, they can copy it.

How to answer in interview: "In Basic Auth, the username and password are joined and turned into a coded text, then sent with every request. This coding is not real protection, so it must always be used with HTTPS."

Why a Safe Connection (HTTPS) Matters

When your browser talks to a website's server, that conversation can travel in two ways: an open way (unsafe) or a locked way (safe). The locked way is called HTTPS.

The "lock" comes from a technology called TLS. It scrambles (mixes up) the data before sending it, so if anyone catches it on the way, they only see meaningless text.

Easy Analogy: Sending data over the open, unsafe way (HTTP) is like mailing a postcard — the postman, and anyone who touches it on the way, can read every word. Sending data over the safe way (HTTPS) is like mailing a locked box — only the person with the right key can open it and read what's inside.

UNSAFE VS SAFE CONNECTION

Client ↔ Server
HTTP — Anyone can read it

Client ↔ Server
HTTPS — Locked and safe

Simple rule: Always use the safe connection (HTTPS) for any real app — especially for login pages, tokens, and payments. Never send passwords over the unsafe connection.

How to answer in interview: "TLS scrambles the data sent between client and server, so no one in between can read it. Every real-world API should run over HTTPS, especially when handling passwords or tokens."

Login Passes (Tokens)

Most apps today don't ask for your username and password on every single request. Instead, they give you a special pass called a token, right after you log in.

This token is just a long piece of text. The app sends this token with every request afterward, to prove "yes, this is a logged-in user."

```
Authorization: Bearer eyJhbGciOiJIUzI1...
```

Easy Analogy: A token is like a wristband at an event. You show your ID card only once, at the entrance (this is the login). They give you a wristband (this is the token). After that, you just show the wristband every time — no need to show your ID card again and again.

In MERN projects: A common way to do this is called JWT (a type of token). After login, the server gives this token to the app. The app saves it and sends it along with every future request.

How to answer in interview: "After login, the server gives the user a token. The app stores this token and sends it with every future request as a Bearer token, instead of sending the username and password again."

CODewithHARDEEP

Liked these notes? **There's more coming.**

This was Video 4 of the "REST API Mastery" series. Next up: Real-world API design — versioning, backward compatibility, and pagination.

- ✓ Subscribe to CodeWithHardeep on YouTube
- ✓ Turn on notifications so you don't miss the next video
- ✓ Comment your doubts — I reply to every comment
- ✓ Watch Video 3 (Core REST Principles) if you missed it

Subscribe to CodeWithHardeep ▶